Lab Cycle III

--16. Write a PL/SQL program to print the series 1 2 3 …. 10

--a) Simple loop

```
set serveroutput on
declare
     n number(2):=1;
begin
loop
            dbms_output.put_line(n);
            n:=n+1;
           if (n > 10) then
              exit;
          end if;
end loop;
end;
/
```

--b) while loop

```
set serveroutput on
declare
     n number(2):=1;
begin
while (n<=10) loop
            dbms_output.put_line(n);
            n:=n+1;
end loop;
end;
/
```

--c) for loop

```
set serveroutput on
declare

begin
for i in 1 .. 10 loop
            dbms_output.put_line(i);
end loop;
end;
/
```

17. Write a PL/SQL program to update the marklist (total and percentage) using a cursor
Table: marks(regno,sname,P1, P2, tot, perc)

Enter 5 set of rows (regno, sname, P1 and P2) and save the changes.
e.g.

```
insert into marks(regno,sname,p1,p2) values(1,'Raj',80,75);
```

--Program

```
select * from marks;
declare
       cursor c1 is select regno,p1,p2 from marks;
       vregno marks.regno%TYPE;
       vp1  marks.p1%TYPE;
       vp2  marks.p2%TYPE;
       vtot  marks.tot%TYPE;
       vperc  marks.perc%TYPE;
begin
      open c1;
      loop
           fetch c1 into vregno, vp1,vp2;
           if c1%NOTFOUND then
               exit;
           end if;
           vtot := vp1+vp2;
           vperc := vtot/2;
          update marks set tot=vtot, perc=vperc where regno=vregno;
          commit;
     end loop;
     close c1;
end;
/
select * from marks;
```

18. Write a PL/SQL program to update the employee details
Table: Employ(empno,ename,basic,hra,da,pf,netsal)

Create the above table and enter values for empno, ename and basic and the program should perform the calculations using a cursor.

DA=40% of basic, HRA=30% of basic, PF=5% of basic
NETSAL=BASIC+DA+HRA-PF

--Program

```
select * from employ;
declare
       cursor c2 is select empno,basic from employ;
       vempno employ.empno%TYPE;
       vbasic  employ.basic%TYPE;
       vhra  employ.hra%TYPE;
       vda employ.da%TYPE;
       vpf  employ.pf%TYPE;
       vnetsal employ.netsal%TYPE;
begin
      open c2;
      loop
           fetch c2 into vempno, vbasic;
           if c2%NOTFOUND then
               exit;
           end if;
```

```
            vhra := vbasic*30/100;
            vda := vbasic*40/100;
            vpf := vbasic*5/100;
           update employ set hra=vhra, da=vda, pf=vpf where empno=vempno;
           commit;
      end loop;
      close c2;
end;
/
select * from employ;
```

--19. Create a new table toppers (regno, sname,tot,perc)
--Write a PL/SQL program to copy the top 3 students (regno, name, total and percentage) from the marks table (Q17) to the toppers table.

--Program

```
delete toppers;
declare
       cursor c3 is select regno,sname,tot, perc from marks order by tot desc;
       vregno marks.regno%type;
       vsname marks.sname%type;
       vtot marks.tot%type;
       vperc marks.perc%type;
       vrecno number(2):=0;
begin
      open c3;
      loop
            fetch c3 into vregno,vsname,vtot,vperc;
            vrecno:=vrecno+1;
            if c3%notfound or vrecno>3 then
                  exit;
            end if;
            insert into toppers values(vregno,vsname,vtot,vperc);
            commit;
      end loop;
end;
/
select * from toppers;
```

--20. Write a PL/SQL program to implement exception handling.(e.g. marks)

```
set serveroutput on
select * from marks;
declare
        vregno marks.regno%type:=&regno;
        vtot marks.tot%type;
        fail exception;
begin
```

```
        select tot into vtot from marks where regno=vregno;
       if vtot < 100 then
          raise fail;
      end if;
 exception
        when no_data_found then
             dbms_output.put_line('Record not found');
        when fail then
             dbms_output.put_line('Not Eligible..');
        when others then
             dbms_output.put_line('Error..');
end;
/
```

21. Write a PL/SQL program to create the following procedures:
a) To add a record (e.g. addproc())
b) To edit a record (e.g. editproc())
c) To delete a record (e.g. delproc())

Table: cust (code, cname, cphone)

```
create or replace procedure addproc(vcode number, vcname varchar,vcphone number) is
begin
        insert into cust values(vcode,vcname,vcphone);
        commit;
end addproc;
/
create or replace procedure editproc(vcode number, vcphone number) is
begin
        update cust set cphone=vcphone where code=vcode;
        commit;
end editproc;
/
create or replace procedure delproc(vcode number) is
begin
       delete from cust where code=vcode;
        commit;
end delproc;
/
```

SQL>@c:\plsql\p21.sql
Procedure created ...

--p21a.sql

```
declare
          vcode number(4):=&cno;
          vcname varchar(20):='&cname';
          vcphone number(10):=&cphone;
begin
```

```
        addproc(vcode,vcname,vcphone);
end;
/
SQL>@c:\plsql\p21a.sql
```

--p21b.sql

```
declare
           vcode number(4):=&cno;
           vcphone number(10):=&cphone;
begin
        editproc(vcode,vcphone);
end;
/
SQL>@c:\plsql\p21b.sql
```

--p21c.sql

```
declare
           vcode number(4):=&cno;
begin
          delproc(vcode);
end;
/
```

SQL>@c:\plsql\p21c.sql
or
SQL> execute addproc(&cno,'&cname',&cphone);
SQL> execute editproc(&cno,&cphone);
SQL> execute delproc(&cno);

SQL> select * from cust;

22. Write a PL/SQL program to create a function to check the availability of the stock.

Table: Stock(itemno, name, qoh,price)

-- p22.sql

```
   create or replace function chkqty(vitemno number,vqty number) return boolean is
   vqoh number(4);
begin
   select qoh into vqoh from stock where itemno=vitemno;
   if(vqoh>=vqty) then
      return true;
   else
     return false;
   end if;
end chkqty;
/
```

SQL>@c:\plsql\p22.sql
Function Created..

```
-- p22a.sql
select * from stock;
declare
   vitemno number(4):=&itemno;
   vqty number(4):=&qty;
begin
  if chkqty(vitemno,vqty) then
        update stock set qoh=qoh-vqty where itemno=vitemno;
        commit;
  else
        raise_application_error(-20002,'item out of stock..');
  end if;
end;
/
select * from stock;
```

23. Program to create a trigger to perform an auditing on cust table. (after update or insert or delete)

Table1: cust (code number(4),cname varchar(20), cphone number(10))
Table2: logtable (uname varchar(20),udate date, action varchar(20))
Trigger: Initiating a set of commands to be executed when some other tasks is performed on a table. It is used for increasing the security and auditing purpose. Once the trigger is created, it will be active and automatically executed based on the timing specified in the code.

```
create or replace trigger trg1 after
    update or insert or delete on cust
declare
    op varchar(10);
begin
     if updating then
          op:='Updated';
    elsif inserting then
         op:='Inserted';
    else
        op:='Deleted';
   end if;
insert into logtable values(user,sysdate,op);
end trg1;
/
```

SQL>@c:\plsql\p23.sql
Trigger created...
SQL> insert into cust values(100,'Ram',9995023456');
SQL> update cust set cphone=9995012345 where code=100;
SQL> select * from cust;
SQL> select * from logtable;

24. Create a trigger to block the delete operation on CUST table
Table: cust (code number(4),cname varchar(20), cphone number(10))

```
create or replace trigger trg2 before delete on cust
declare
begin
  raise_application_error(-20002,'Invalid Command');
end;
/
```

SQL>@c:\plsql\p24.sql
Trigger created...

SQL> delete from cust where code=100;
Error-Invalid Command..

25. Create a trigger to move the deleted records to another table
Table1: cust (code number(4),cname varchar(20), cphone number(10))
Table2: oldcust (code number(4),cname varchar(20), cphone number(10))

```
create or replace trigger trg3 after delete on cust
declare
begin
  insert into oldcust values(:old.code,:old.cname,:old.cphone);
end;
/
```

SQL>@c:\plsql\p25.sql
Trigger created...

SQL> delete from cust where code=100;
SQL> select * from cust;
SQL> select * from oldcust;

****